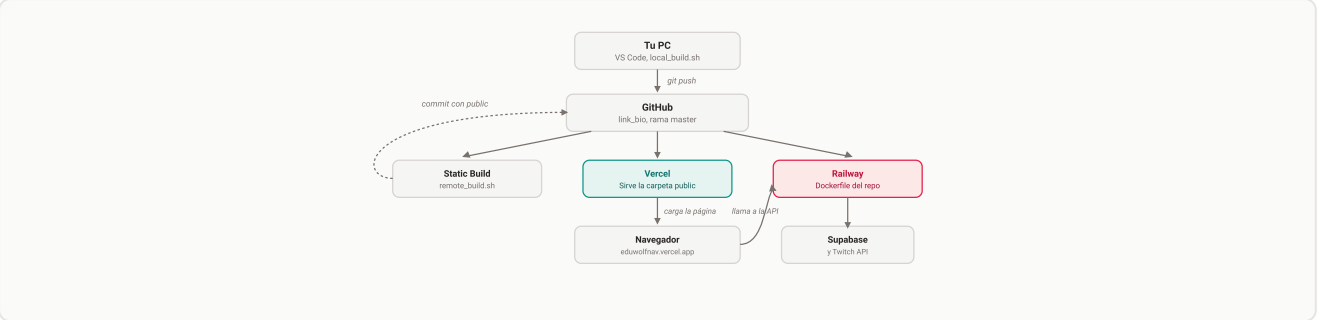
Cómo se despliega link_bio

GitHub, Vercel y Railway: del código en tu PC a la web en producción

Resumen

`link_bio` se publica con tres servicios que trabajan juntos. GitHub guarda el código y es el punto de partida de todo. Vercel publica el frontend, la página que ve el usuario. Railway publica el backend, la API. Entre medio, una acción de GitHub llamada Static Build prepara el frontend estático antes de que Vercel lo sirva.

Diagrama del flujo



Verde azulado: frontend. Coral: backend. Gris: control de versiones, automatización y cliente.

Flujo paso a paso

1. Haces cambios en tu PC (VS Code) y los subes con `git push origin master`. Opcionalmente, `local_build.sh` genera el build estático en tu equipo, igual que lo hace la acción.
2. GitHub recibe el push y avisa a tres piezas en paralelo: Static Build, Vercel y Railway.
3. Static Build (`.github/workflows/remote_build.yml`) corre en un servidor Ubuntu con Python 3.13. Define `REFLEX_API_URL=https://eduwolfnav.up.railway.app` y ejecuta `remote_build.sh`: instala dependencias, borra `public`, corre `reflex init` y `reflex export --frontend-only`, y extrae `frontend.zip` dentro de `public`.
4. La acción hace un commit automático con el mensaje *Update static build* que incluye la carpeta `public` y lo sube a `master`.

5. Ese commit nuevo vuelve a avisar a Vercel y Railway. Vercel publica el frontend estático actualizado.
6. Railway construye y publica el backend desde el Dockerfile del repo.
7. En producción, el navegador carga la página desde `eduwolfnav.vercel.app` y esta llama a la API en `eduwolfnav.up.railway.app`. El backend usa Supabase y la API de Twitch (según `SupabaseAPI.py` y `TwitchAPI.py`).

Qué hace cada servicio

Servicio	Rol	Se activa con	Resultado
GitHub	Control de versiones y punto de partida	Tu `git push` a `master`	Repo `eduwolfnav/link_bio`
GitHub Actions (Static Build)	Construye el frontend estático	Push a `master`	Commit *Update static build* con `public`
Vercel	Hosting del frontend	Commits en el repo	`eduwolfnav.vercel.app`
Railway	Hosting del backend (API)	Commits en el repo	`eduwolfnav.up.railway.app`

Recomendaciones

- Haz `git pull` antes de tu siguiente `git push`. La acción agrega un commit a `master` después de cada push, así que tu repo local queda atrasado y Git puede rechazar el push.
- Configura «**Watch Paths**» en Railway para ignorar `public/**`. El commit automático solo cambia esa carpeta, así que el backend no necesita reiniciarse innecesariamente.
- Si el último despliegue de Railway no aparece como activo, usa **Redeploy** desde su panel.
- Mantén el `.env` fuera de GitHub (ya está en `.gitignore`) y carga las variables sensibles desde los paneles de Railway y Vercel.

Archivos clave del proyecto

Archivo	Para qué sirve
`local_build.sh`	Genera el build estático en tu PC (activa `.venv`, instala dependencias, exporta el frontend).
`remote_build.sh`	Misma tarea, pero la ejecuta la acción de GitHub en el servidor.
`.github/workflows/remote_build.yml`	Workflow Static Build: corre en cada push a master y hace el commit automático.
`rxconfig.py`	Configuración de Reflex: `api_url` hacia Railway y CORS para `localhost` y Vercel.
`Dockerfile`	Receta de construcción del backend.
`link_bio/api/`	API del backend: `api.py`, `SupabaseAPI.py`, `TwitchAPI.py`.
`public/`	Frontend estático generado; es lo que sirve Vercel.

Supuestos confirmados

- ✓ **Vercel y Railway están conectados directamente al repo de GitHub:** Confirmado mediante la revisión de los paneles de administración de Vercel y Railway. Ambos están vinculados a `eduwolfnav/link_bio` en la rama `master`.
- ✓ **Railway construye el backend con el Dockerfile del repo:** Confirmado en la sección *Build* de Railway, donde se autodetectó y configuró la compilación usando el `Dockerfile`.
- ✓ **Supabase es la base de datos del backend:** Confirmado en el panel de variables de entorno de Railway, donde están configuradas las variables `SUPABASE_URL` y `SUPABASE_PUBLISHABLE_KEY`.